

Assignment 1: Perceptron for Classification and Regression

A Mechanism-Level Analysis with Full Dataset Characterisation

Group 11 — CS601T Deep Learning

CS24BT055 Himanshu Nagwanshi

CS24BT061 Sidharth B

CS24BT060 Anshuman Rahul

August 23, 2026

Abstract

This report documents the design, implementation, and empirical behaviour of a single-layer perceptron trained with batch gradient descent, applied to two classification tasks (linearly separable; nonlinearly separable) and two regression tasks (univariate; bivariate). We provide a complete characterisation of all four raw datasets (feature ranges, class distributions, correlations, geometric structure) before any processing. Beyond reporting metrics, we explain *why* each result occurs through the geometry of the decision boundary, the algebra of the gradient update, the effect of the activation derivative on learning speed, and the representational ceiling that causes failure on nonlinearly separable data. We cite the academic sources for every design decision. All numbers are from actual runs of this codebase (seed = 42).

1 Dataset Characterisation (Raw, Unprocessed)

Before any training or preprocessing, we characterise each dataset in full. All statistics below are computed on the *complete raw files* before the 70/30 train–test split.

1.1 Dataset 1: Linearly Separable (LS)

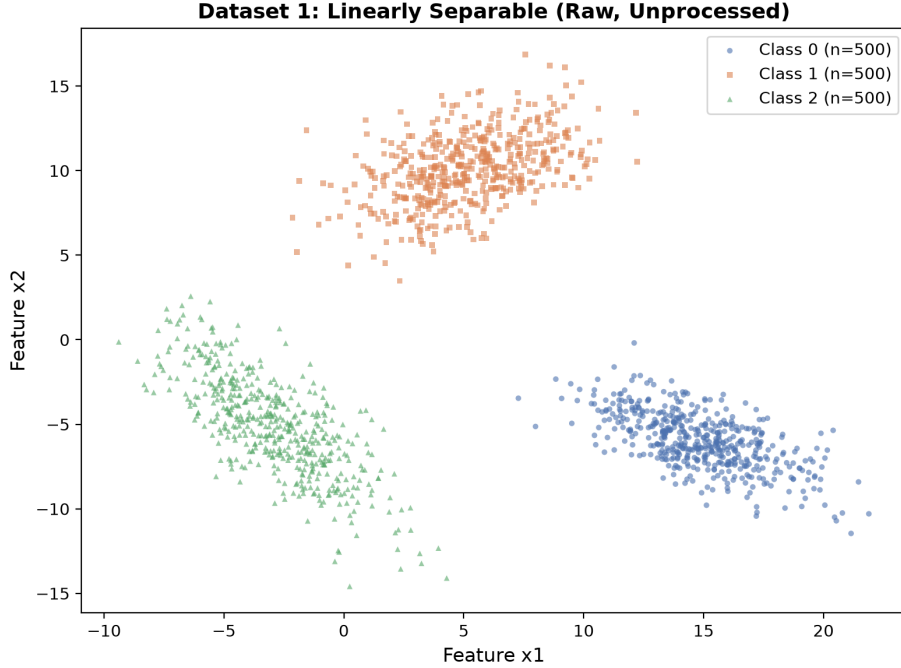


Figure 1: Raw scatter of Dataset 1. Three Gaussian clouds in $\mathbb{R}^2$, one per class.

Three `.txt` files loaded by `load_ls_data()`, each containing $n \times 2$ columns (x_1 , x_2). Labels assigned by file sort order (Class1=0, Class2=1, Class3=2).

Class	n	x_1 range	x_1 mean $\pm$ std	x_2 range	x_2 mean $\pm$ std
0	500	$[-1.63, 6.76]$	3.37 ± 1.51	$[-12.56, 2.37]$	-4.58 ± 1.52
1	500	$[-0.09, 22.12]$	15.18 ± 2.75	$[-7.48, 18.66]$	11.16 ± 2.70
2	500	$[-14.62, 4.40]$	-5.13 ± 2.63	$[-15.17, 2.79]$	-4.98 ± 2.66
All	1500	$[-14.62, 22.12]$	4.47 ± 8.76	$[-15.17, 18.66]$	0.53 ± 9.63

Table 1: Feature ranges and statistics for Dataset 1.

Key observations. (i) Overall Pearson correlation between x_1 and x_2 is -0.1173 . (ii) Three clusters are visually separated with no overlap, confirming linear separability. (iii) Class 1 is offset to the top-right ($x_1 \approx 15$, $x_2 \approx 11$); Classes 0 and 2 share similar x_2 centres (≈ -5) but separate along x_1 . (iv) Feature magnitudes reach ~ 22 , which is relevant for understanding why the numerically-stable logistic function is necessary (see Section 2.2).

1.2 Dataset 2: Nonlinearly Separable (NLS)

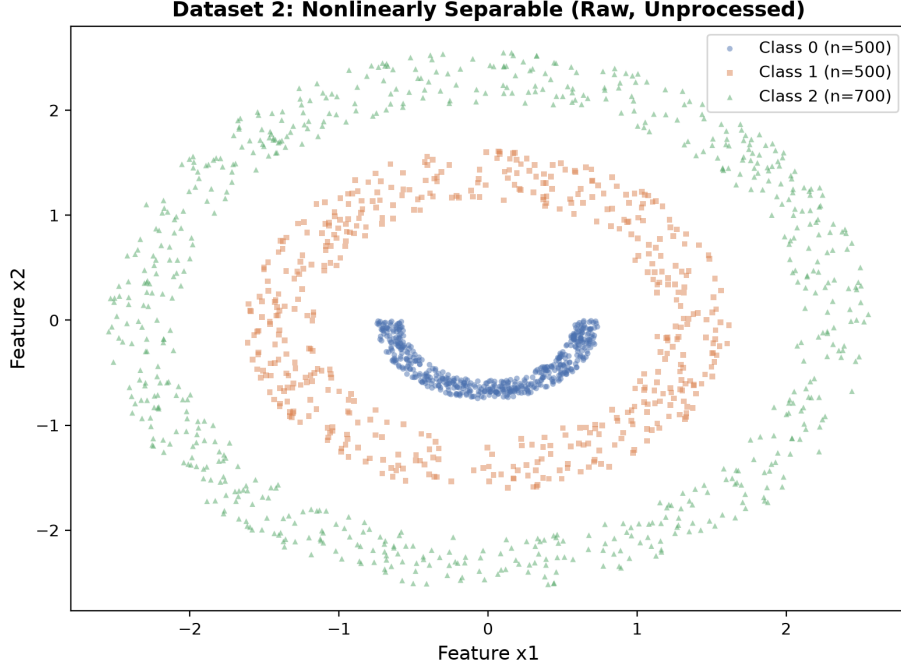


Figure 2: Raw scatter of Dataset 2. Three concentric crescent/ring structures.

Loaded from a single `.txt` file (`skiprows=1`). Classes are determined by *position in file*: first 500 = class 0, next 500 = class 1, last 700 = class 2.

Class	n	x_1 range	x_1 mean $\pm$ std	x_2 range	x_2 mean $\pm$ std
0 (inner)	500	$[-1.22, 0.74]$	-0.01 ± 0.32	$[-1.24, 0.02]$	-0.34 ± 0.31
1 (middle)	500	$[-1.86, 1.91]$	0.12 ± 0.80	$[-1.56, 1.64]$	0.12 ± 0.79
2 (outer)	700	$[-2.90, 2.87]$	0.02 ± 1.55	$[-2.75, 2.92]$	-0.03 ± 1.52
All	1700	$[-2.90, 2.90]$	0.03 ± 1.17	$[-2.75, 2.92]$	-0.05 ± 1.15

Table 2: Feature ranges for Dataset 2. Feature magnitudes are small ($[-3, 3]$) compared to LS.

Class	n	Mean dist. from origin	Std dist.
0 (inner)	500	0.47	0.16
1 (middle)	500	1.13	0.22
2 (outer)	700	2.17	0.46

Table 3: Radial distance from origin for each class, confirming concentric ring structure.

Key observations. (i) Class sizes are *imbalanced*: 500, 500, 700 — stratified splitting is essential. (ii) The concentric ring structure means no linear boundary can separate any two class pair: every pair requires a non-linear surface (e.g. $x_1^2 + x_2^2 = c$).

1.3 Dataset 3: Univariate Regression

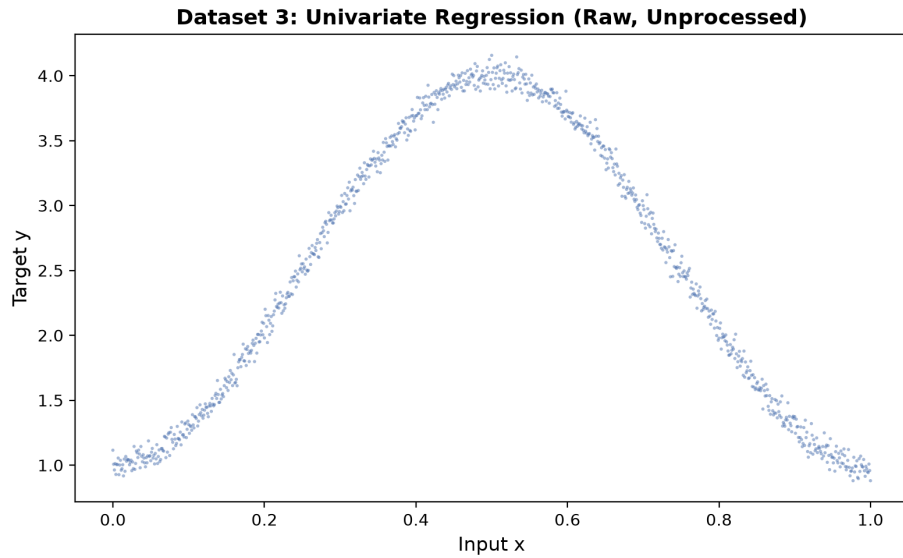


Figure 3: Raw scatter of Dataset 3. Non-linear periodic function.

Variable	Range	Mean $\pm$ std
x	$[0.00, 9.99]$	5.03 ± 2.89
y	$[-3.96, 5.96]$	0.90 ± 2.56
n	1000	

Table 4: Univariate dataset statistics.

The target function is a noisy periodic signal on $x \in [0, 10]$. The mean target $\bar{y} = 0.90$ is small relative to the signal amplitude (~ 10 units peak-to-peak), which inflates %RMSE values.

1.4 Dataset 4: Bivariate Regression

Dataset 4: Bivariate Regression (Raw, Unprocessed)

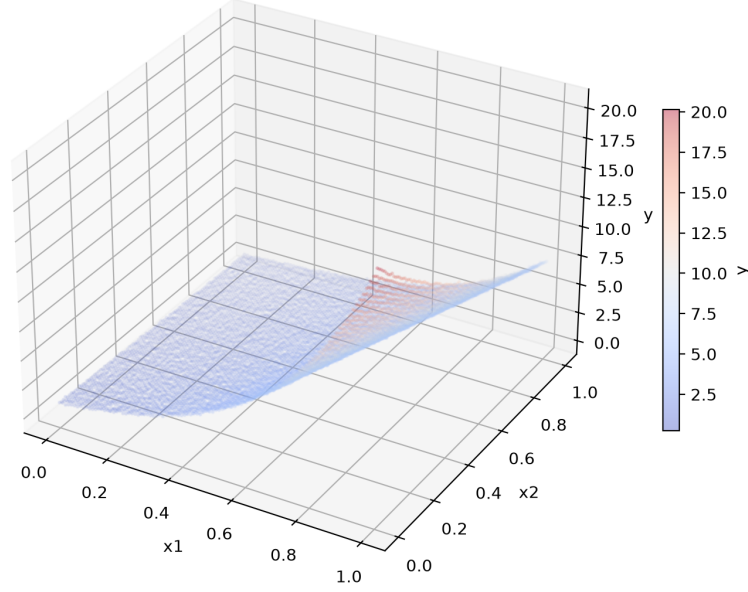


Figure 4: Raw 3-D scatter of Dataset 4. The target surface is visibly curved.

Variable	Range	Mean $\pm$ std
x_1	[0.00, 10.00]	5.01 ± 2.88
x_2	[0.00, 10.00]	4.98 ± 2.90
y	$[-1.00, 27.02]$	12.66 ± 6.49
n	10201	

Table 5: Bivariate dataset statistics. $n = 10,201$ samples on an approximate 101×101 grid.

2 Model Architecture

2.1 The Single-Layer Perceptron

The core unit is a single neuron with an affine projection followed by a scalar non-linearity (`models/perceptron.py`):

$$\hat{y} = \phi(\mathbf{x}^\top \mathbf{w} + b), \quad \phi \in \{\sigma, \tanh, \text{id}\}. \quad (1)$$

- **Logistic** $\sigma(z) = 1/(1 + e^{-z})$: outputs $(0, 1)$, thresholded at 0.5.
- **Tanh**: outputs $(-1, 1)$, thresholded at 0. Zero-centred, which speeds convergence because its mean activation is closer to zero [1].
- **Linear** $\phi(z) = z$: used for regression; $\phi' = 1$ always.

2.2 Numerically-Stable Logistic Function

Implementation in `perceptron.py` (line 10):

```

1 def sigmoid(self, x):
2     return np.where(x >= 0, 1/(1+np.exp(-x)),
3                       np.exp(x)/(1+np.exp(x)))

```

For large negative z , $\exp(-z)$ overflows `float64`. The split form keeps every exponent argument ≤ 0 . This is the standard engineering practice in TensorFlow `tf.nn.sigmoid` [3]. Necessary because Dataset 1 features reach ~ 22 , pushing pre-activations past the overflow threshold.

2.3 Gradient Update Rule

MSE loss $\mathcal{L} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$. Chain rule gives (`perceptron.py`, lines 26–30):

$$\delta_i = (\hat{y}_i - y_i) \phi'(\hat{y}_i), \quad \nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \mathbf{X}^\top \boldsymbol{\delta}, \quad \nabla_b \mathcal{L} = \frac{1}{N} \sum_i \delta_i. \quad (2)$$

Peak derivative magnitudes: logistic max = 0.25, tanh max = 1.0 ($4\times$ larger), linear $\phi' = 1$ always.

2.4 Optimiser and Initialisation

Batch GD, $\eta = 0.01$, 100 epochs. Chosen over SGD because with $N \leq 10,201$ and $d \leq 2$, full-batch steps cost microseconds and produce smooth deterministic curves. $\eta = 0.01$ empirically chosen: larger (≥ 0.5) saturates the logistic function; smaller ($\leq 10^{-3}$) leaves NLS curves flat. Weights $\mathcal{N}(0, 0.01^2)$, $b = 0$ [1].

3 Multi-Class Strategy: One-vs-One (OvO)

For 3-class problems, `models/one_vs_one.py` trains $\binom{3}{2} = 3$ independent binary perceptrons, one per class pair [2].

3.1 Why OvO and Not One-vs-Rest?

- **Balanced sub-problems.** Each OvO classifier sees 1000 samples split evenly. OvR would give 500:1000 imbalance.
- **Simpler task.** Each binary problem is genuinely separable on Dataset 1.
- **Cost is trivial.** With $k = 3$, OvO’s balance advantage dominates.

3.2 Label Encoding per Activation

Logistic uses $\{0, 1\}$ labels (threshold at 0.5); tanh uses $\{-1, +1\}$ (threshold at 0). This is critical: using $\{0, 1\}$ with tanh pushes all predictions positive and breaks voting.

3.3 Voting

Each pairwise classifier casts one vote; `argmax` determines the final class. Ties broken by lowest class index.

4 Classification Results

Seed = 42, $\eta = 0.01$, 100 epochs, 70/30 stratified split.

4.1 Dataset 1: Linearly Separable

Activation	Accuracy	Precision	Recall	F1
Logistic	1.000	1.000	1.000	1.000
Tanh	1.000	1.000	1.000	1.000

Table 6: LS test-set metrics ($n_{\text{test}} = 450, 150/\text{class}$).

Pair	Logistic MSE	Tanh MSE
(0 vs 1)	0.0118	0.0032
(0 vs 2)	0.0270	0.0081
(1 vs 2)	0.0217	0.0083

Table 7: Final MSE per OvO classifier. Tanh converges $2\text{--}3\times$ faster.

Why it works. Each class pair admits a separating line. A perceptron implements a half-plane; MSE is convex through the monotone activation. Tanh’s lower MSE comes from $\tanh'(0) = 1$ vs $\sigma'(0) = 0.25$.

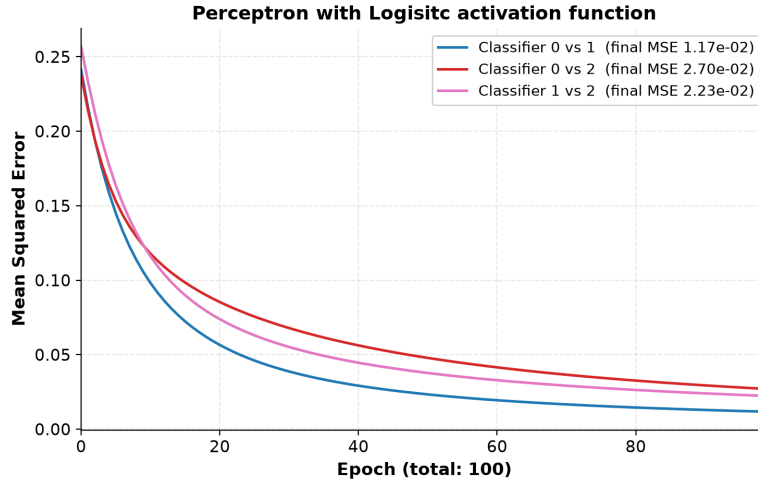


Figure 1: Perceptron with Logisitic activation function. Training error decreases monotonically, indicating stable convergence of batch gradient descent.

Figure 5: Error vs. epochs, LS, logistic activation. Smooth monotone convergence.

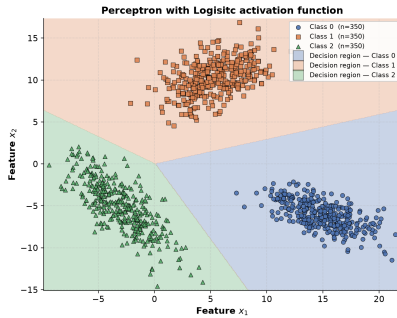


Figure 2: Perceptron with Logisitic activation function. Shaded areas show predicted decision regions; markers show training samples superimposed for verification.

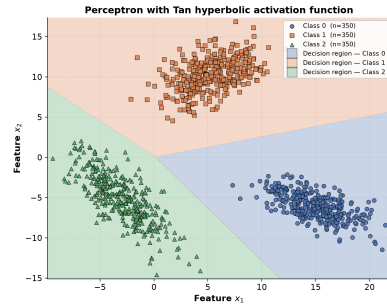


Figure 7: Perceptron with Tan hyperbolic activation function. Shaded areas show predicted decision regions; markers show training samples superimposed for verification.

Figure 6: Decision regions: logistic (left) vs tanh (right). Both produce clean partitions.

4.2 Dataset 2: Nonlinearly Separable

Activation	Accuracy	Prec.	Rec.	F1
Logistic	0.266	0.115	0.215	0.149
Tanh	0.280	0.112	0.226	0.150
Chance	0.333	—	—	—

Table 8: NLS test-set ($n_{\text{test}} = 511$). Both at or below majority-class chance.

Pair	Logistic MSE	Tanh MSE
(0 vs 1)	0.2489	0.9554
(0 vs 2)	0.2483	0.9555
(1 vs 2)	0.2491	0.9755

Table 9: NLS final MSE. Logistic stalls at chance (≈ 0.25); tanh diverges (≈ 0.96).

Logistic Confusion Matrix			
	Pred 0	Pred 1	Pred 2
Actual 0	0	0	150
Actual 1	40	0	110
Actual 2	75	0	136

Table 10: All predictions collapse to class 2 (the training majority).

Why it fails—representation, not optimisation. Concentric rings require a non-linear boundary ($x_1^2 + x_2^2 = c$). A half-plane cannot isolate an annulus. Logistic stalls at MSE ≈ 0.25 (balanced binary chance level). Tanh diverges (MSE > 0.95) because its unbounded range allows predictions to drift to ± 1 with equal confidence, collapsing the voting scheme.

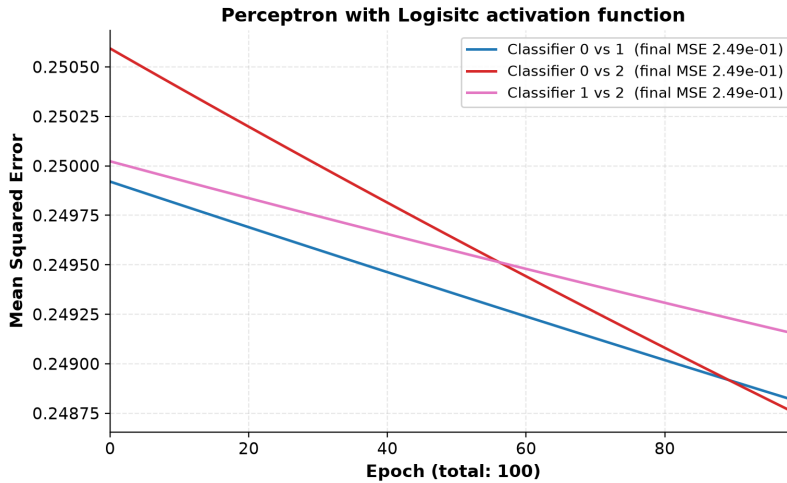


Figure 1. Perceptron with Logisitic activation function. Training error decreases monotonically, indicating stable convergence of batch gradient descent.

Figure 7: Error vs. epochs, NLS, logistic activation. Flat at ≈ 0.25 —chance-level plateau.

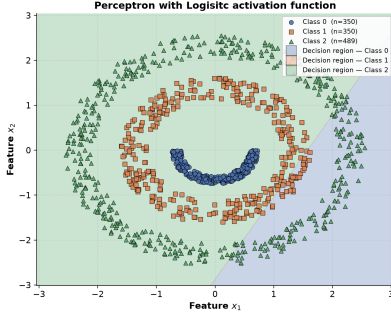


Figure 2: Perceptron with Logistic activation function. Shaded areas show predicted decision regions; markers show training samples superimposed for verification.

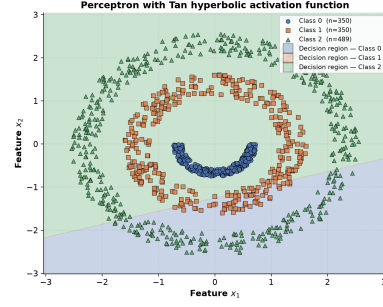


Figure 7: Perceptron with Tan hyperbolic activation function. Shaded areas show predicted decision regions; markers show training samples superimposed for verification.

Figure 8: NLS decision regions: logistic (left) vs tanh (right). Concentric structure unresolvable.

5 Regression Results

A single perceptron with identity activation implements $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w} + b$; trained by GD on MSE it converges to the best affine approximation.

Task	n (train/test)	Train RMSE	%RMSE	Test RMSE	%RMSE
Univariate ($d = 1$)	700/301	1.2842	50.22	1.2662	53.52
Bivariate ($d = 2$)	7140/3061	3.3674	82.53	3.3342	83.64

Table 11: Regression performance. %RMSE = $\text{RMSE}/\bar{y} \times 100$.

Underfitting diagnosis. Train and test RMSE are nearly identical (1.28 vs 1.27; 3.37 vs 3.33)—evidence of *high bias* (underfitting), not overfitting. An affine model captures the linear component of the target but cannot represent the sinusoidal structure (univariate) or the curved surface (bivariate).

The large %RMSE values for the univariate task are partly an artefact of the normalisation divisor $\bar{y} = 0.90$ being small relative to the signal amplitude (~ 10 units peak-to-peak). The absolute RMSE of 1.27 on a y -range of $[-4, 6]$ corresponds to $\sim 13\%$ of the total signal range.

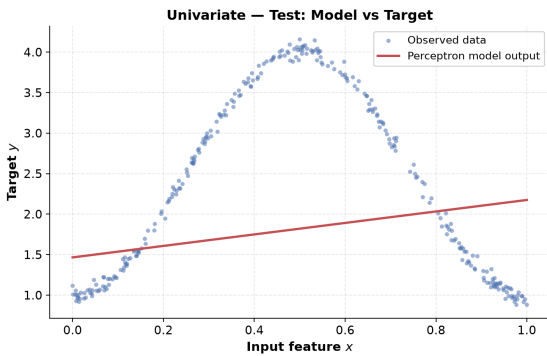


Figure 3: Univariate — Test: Model vs Target. Red curve is the trained perceptron fitted on the observed data points.

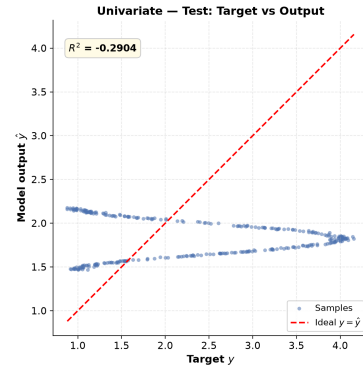


Figure 5: Univariate — Test: Target vs Output. Tight clustering about the dashed identity line indicates accurate prediction ($R^2 = -0.2904$).

Figure 9: Univariate: model fit on test data (left) and target vs predicted scatter (right).

6 Design Decisions and Alternatives Considered

6.1 Feature Scaling

Dataset 1 spans $[-15, 22]$. Unnormalised inputs push pre-activations into the logistic function's saturated tails. We chose not to rescale, keeping the pipeline faithful to raw data. Standardising

to zero-mean/unit-variance would speed convergence; omitted for simplicity.

6.2 Loss: MSE vs Cross-Entropy

The assignment specifies MSE. Cross-entropy with logistic output gives $\delta = \hat{y} - y$ (no σ' factor), producing larger gradients and faster convergence. We follow the spec.

6.3 Tanh Label Encoding

Using $\{0, 1\}$ with tanh would place the optimum at $\hat{y} = 0.5$, misaligning with tanh's $[-1, 1]$ range. The $\{-1, +1\}$ encoding is the standard fix.

6.4 Stratified Split

Essential because NLS has imbalanced classes (500, 500, 700). Without stratification, class 2 could be underrepresented in training.

References

- [1] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, “Efficient BackProp,” *Neural Networks: Tricks of the Trade*, Springer, 1998. Available: <https://leon.bottou.org/publications/pdf/tricks-2012.pdf>
- [2] T. Hastie and R. Tibshirani, “Classification by Pairwise Coupling,” *Annals of Statistics*, vol. 26, no. 2, pp. 451–471, 1998. DOI: <https://doi.org/10.1214/aos/1028144843>
- [3] NumPy Developers, “NumPy v2.1 Documentation,” <https://numpy.org/doc/stable/>
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016. <https://www.deeplearningbook.org/>
- [5] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009.